



南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

Big Data Analysis Software and Application (Hadoop or Spark)

Lecture 12: Introduction to Ray



陈冠华 CHEN Guanhua

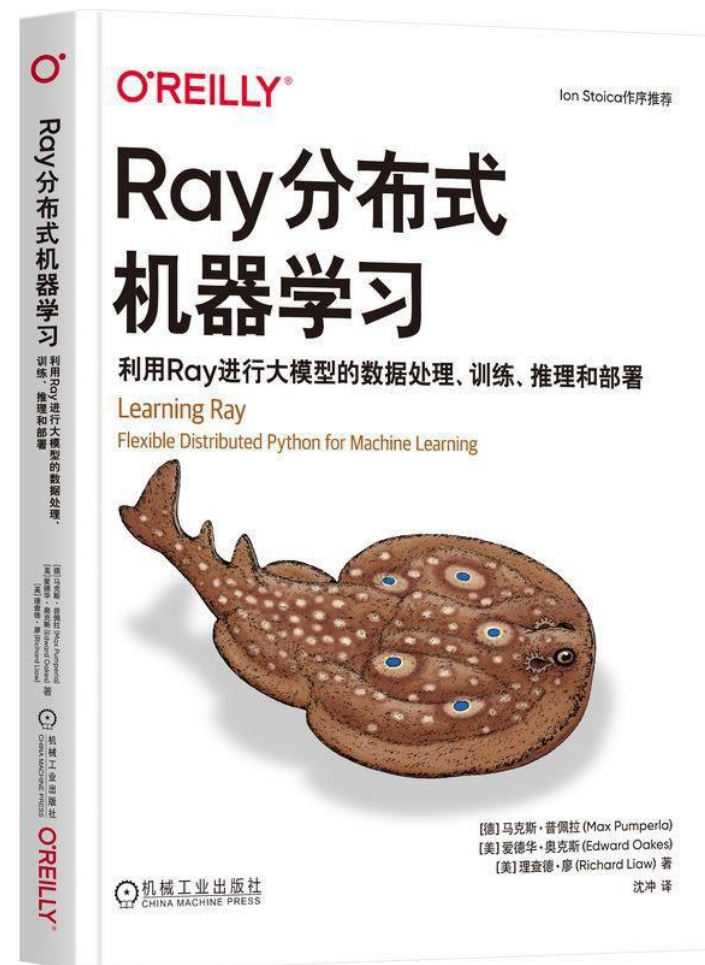
Department of Statistics and Data Science

Content

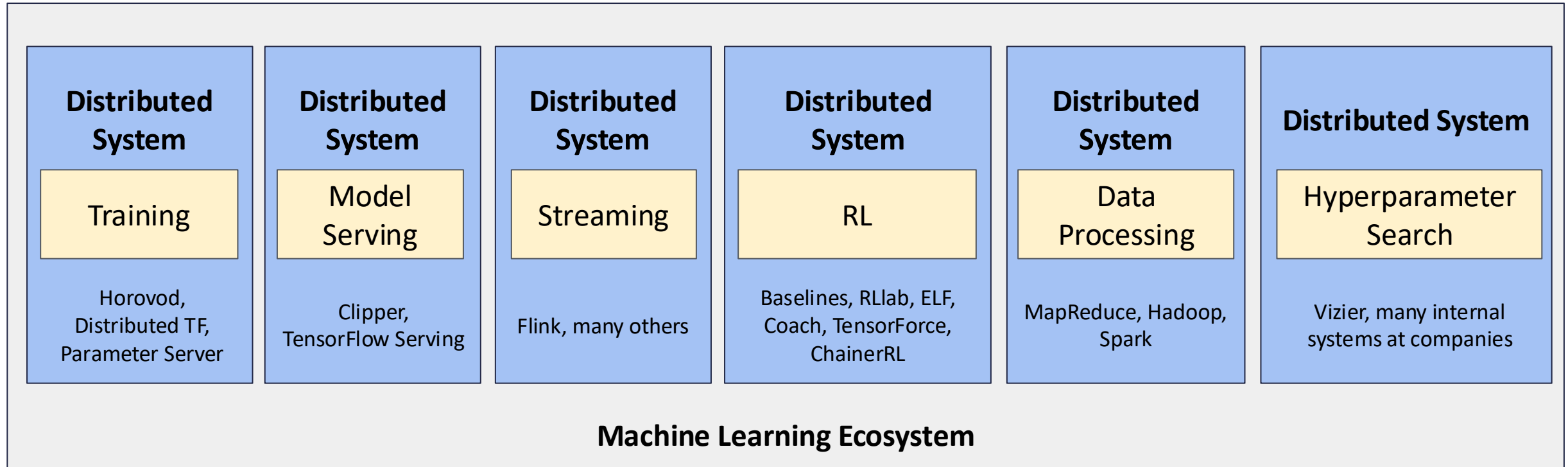


南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

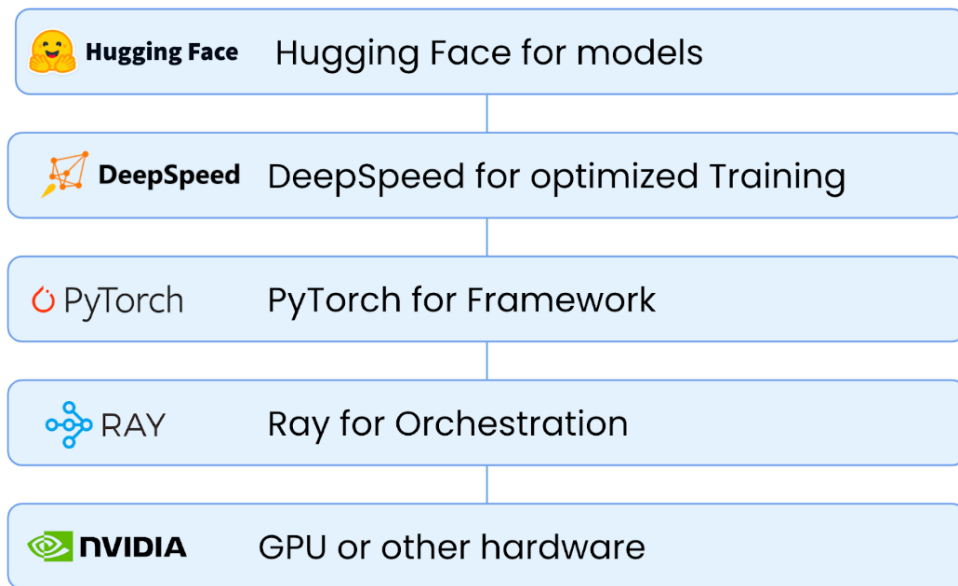
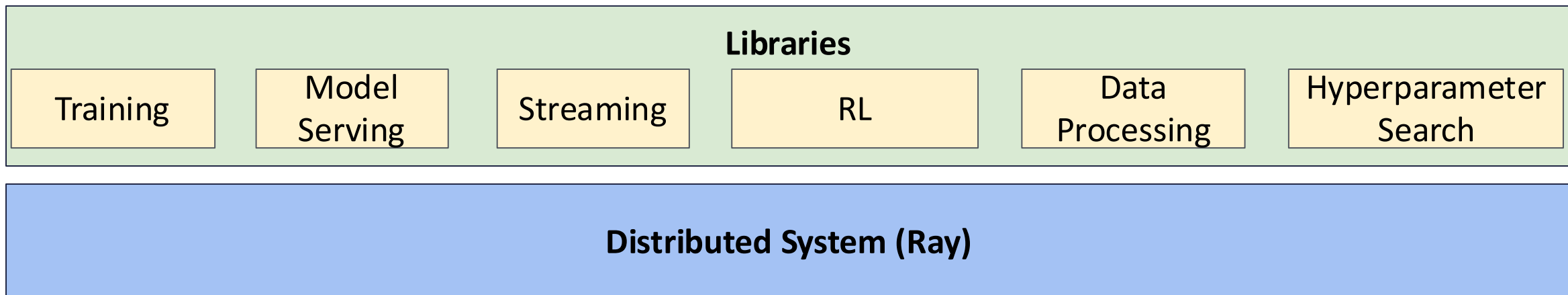
- Introduction
- Basic Concepts
- Ray Train
- Deployment
- Conclusion



The Machine Learning Ecosystem



Introduction



2023-01-04 11:18 北京 来源：澎湃新闻·澎湃号·湃客



字号▼

机器之心报道

编辑：泽南、小舟

Ray 被 OpenAI、亚马逊等科技公司用来开发大模型，是最近异军突起的框架。

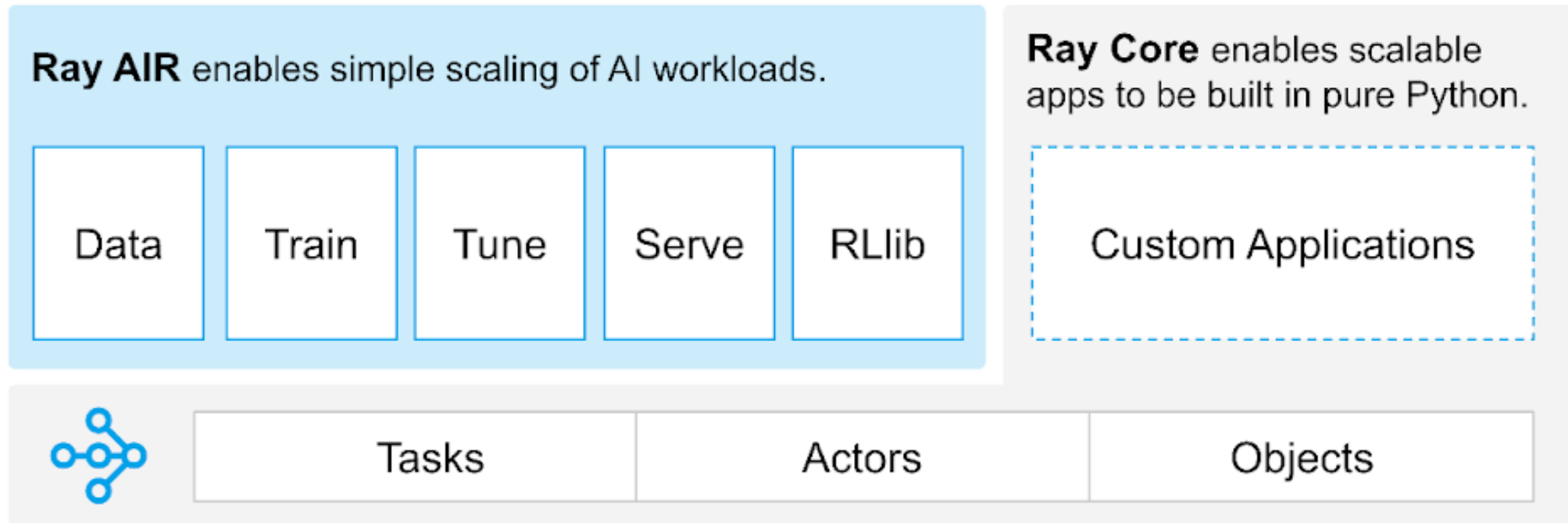
最近一段时间，文本生成的人工智能在互联网上掀起了一阵风暴：ChatGPT 因为可以对人们能想到的几乎任何问题提供非常详细、近乎逼真的回答而受到追捧。大模型应用的出现让人们对于 AI 技术突破充满了信心，不过很少有人知道在其背后，一个分布式机器学习框架正为这场生成式 AI 革命提供动力。

分布式计算框架 Ray 来自 A16z 支持的初创公司 Anyscale，它是使 OpenAI 能够强化其训练 ChatGPT 等模型的关键。在 OpenAI 最近的所有大型语言模型背后都有 Ray 的身影——它可能也会是 OpenAI 备受期待的 GPT-4 背后的框架。随着大模型技术的不断落地，业内人士认为通过生成接近人类的内容，一个价值数十亿美元的产业正在形成。

[How OpenAI Uses A16z-Backed Anyscale's Ray to Train Tools Like ChatGPT - Business Insider](#)

Introduction

- Provide a universal API for distributed computing for deep learning
- Support for full ML lifecycle (data collection, training, RL simulation, serving)



[Getting Started — Ray 2.46.0](#)

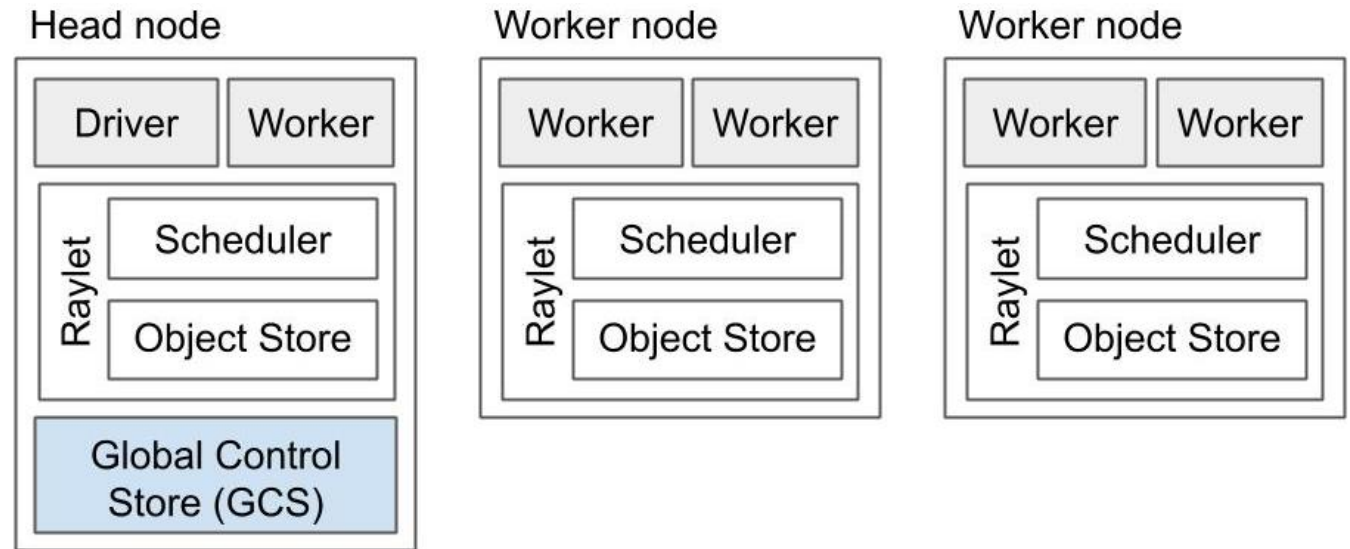
[Ray Use Cases — Ray 2.46.0](#)

Basic Concepts

- Ray task
 - The unit of work scheduled by Ray
 - Correspond to **python function/method**
- Ray actor
 - Can have inner states
 - Correspond to **python class**
- Objects
 - Remote object can be stored **anywhere** in a Ray cluster
 - Remote object can live on one or many nodes
 - Use `ray.get()` method to fetch the result of a remote object from an **object ref**

Basic Concepts

- Shared-memory object store
 - Share objects efficiently between workers without creating copies
- In-memory database
 - Storing metadata needed to rerun tasks in the event of machine failures
- Ray Cluster
 - Consists of a single head node and any number of connected worker nodes



Global Control Service (GCS) is a server that

- Manage cluster-level metadata
- Manage actor, placement groups and node

[User Guides — Ray 2.46.0](#)

- Parallelism with Tasks
 - Initialize with `ray.init()`
 - Decorate with `@ray.remote` to run this function remotely
 - Call the function with `.remote()` (Get a **future**).
 - Yield an **object reference**, that can be fetched with `ray.get()`.

- Future: a reference to the eventual output
- The actual function execution will take place **in the background**

```
import ray
import time
ray.init()

@ray.remote(num_cpus=4, num_gpus=2)
def f(x):
    time.sleep(1)
    return x

# Start 4 tasks in parallel.
result_ids = []
for i in range(4):
    result_ids.append(f.remote(i))

# Wait for the tasks to complete
# and retrieve the results.
# With at least 4 cores, this will
# take 1 second.
results = ray.get(result_ids) #
[0, 1, 2, 3]
```

- Task Dependencies
 - Tasks can also depend on other tasks
 - The outputs of the first two tasks will **automatically** be passed as arguments into the third task
 - Tasks can be composed with arbitrary DAG dependencies

```
# However, you can configure Ray tasks to return multiple Object Refs.
@ray.remote(num_returns=3)
def return_multiple():
    return 0, 1, 2

object_ref0, object_ref1, object_ref2 = return_multiple.remote()
assert ray.get(object_ref0) == 0
assert ray.get(object_ref1) == 1
assert ray.get(object_ref2) == 2
```

```
import ray
import numpy as np
ray.init()

@ray.remote
def create_matrix(size):
    return np.random.normal(size=size)

@ray.remote
def multiply_matrices(x, y):
    return np.dot(x, y)

x_id = create_matrix.remote([1000, 1000])
y_id = create_matrix.remote([1000, 1000])
z_id = multiply_matrices.remote(x_id, y_id)

# Get the results.
z = ray.get(z_id)
```

Ray actors

- A process that runs somewhere in the cluster and holds a copy of the object
- Allow **mutable state** to be shared between multiple tasks
 - Methods called on different actors can execute **in parallel**
 - Methods called on the same actor are executed **serially** in the order that they are called
 - Methods on the same actor will **share state** with one another

```
import ray
ray.init()

@ray.remote
class Counter(object):
    def __init__(self):
        self.n = 0

    def increment(self):
        self.n += 1

    def read(self):
        return self.n

counters = [Counter.remote() for i in range(4)]
[c.increment.remote() for c in counters]
futures = [c.read.remote() for c in counters]
print(ray.get(futures)) # [1, 1, 1, 1]
```

Ray actors

- Pass around handles to an actor, which allows other actors or other tasks to all invoke methods on the same actor.

```
import ray
import numpy as np
ray.init()
import time

@ray.remote
class MessageActor(object):
    def __init__(self):
        self.messages = []

    def add_message(self, message):
        self.messages.append(message)

    def get_and_clear_messages(self):
        messages = self.messages
        self.messages = []
        return messages
```

```
# Define a remote function which loops around and pushes
# messages to the actor.
@ray.remote
def worker(message_actor, j):
    for i in range(100):
        time.sleep(1)
        message_actor.add_message.remote(
            "Message {} from worker {}".format(i, j))

# Create a message actor.
message_actor = MessageActor.remote()

# Start 3 tasks that push messages to the actor.
[worker.remote(message_actor, j) for j in range(3)]

# Periodically get the messages and print them.
for _ in range(100):
    new_messages =
ray.get(message_actor.get_and_clear_messages.remote())
    print("New messages:", new_messages)
    time.sleep(1)
```

[Actors — Ray 2.46.0](#)

Examples: Batch Prediction with Ray Core

- Process
 - Let the driver load the model
 - Let the driver `ray.put(model)` to store the model into object store
 - Pass the same **reference** of the model to each task
 - Each worker executes predictions on the assigned shard, and writes out results
- **Task-based** batch prediction
 - Each task takes in the model reference and a shard of input dataset
- **Actor-based** batch prediction
 - Use actor to fetch the model just once and reuse it for all tasks assigned to the actor

[Batch Prediction with Ray Core — Ray 2.46.0](#)

Examples: Batch Prediction with Ray Core

- Task-based Batch Prediction

```
import torch

@ray.remote(num_gpus=1)
def make_prediction(model: torch.nn.Module,
                    shard_path):
    model.to(torch.device("cuda"))
    inputs =
    pq.read_table(shard_path).to_pandas().to_numpy()

    results = []
    for item in inputs:
        results.append(model(item))
    return results
```

```
input_files = [
    f"s3://anonymous@air-example-
data/fe41422b01c04169af2a65a83b753e0f_{i:06d}.parquet"
    for i in range(12)
]

model = load_model()
model_ref = ray.put(model)

result_refs = []
for file in input_files:
    result_refs.append(make_prediction.remote(model_ref, file))
results = ray.get(result_refs)
```

[Batch Prediction with Ray Core — Ray 2.46.0](#)

Examples: Batch Prediction with Ray Core



- Actor-based Batch Prediction with Ray Core

```
import pandas as pd
import pyarrow.parquet as pq
import ray

@ray.remote
class BatchPredictor:
    def __init__(self, model):
        self.model = model

    def predict(self, shard_path):
        df = pq.read_table(shard_path).to_pandas()
        result = self.model(df)
        return len(result)
```

```
from ray.util.actor_pool import ActorPool
model = load_model()
model_ref = ray.put(model)
num_actors = 4
actors = [BatchPredictor.remote(model_ref) for _ in
           range(num_actors)]
pool = ActorPool(actors)
input_files = [
    f"s3://anonymous@air-example-
data/fe41422b01c04169af2a65a83b753e0f_{i:06d}.parquet"
    for i in range(12)
]

for file in input_files:
    pool.submit(lambda a, v: a.predict.remote(v), file)

while pool.has_next():
    print("Prediction output size:", pool.get_next())
```

Example: Speed Up the Web Crawler

- Build a simple web scraper in Python and parallelize it with Ray Tasks

```
import requests
from bs4 import BeautifulSoup

def find_links(start_url, base_url, depth=2):
    if depth == 0:
        return set()

    page = requests.get(start_url)
    soup = BeautifulSoup(page.content, "html.parser")
    elements = soup.find_all("a", href=True)
    links = extract_links(elements, base_url)

    for url in links:
        new_links = find_links(url, base_url, depth-1)
        links = links.union(new_links)
    return links
```

```
import ray

@ray.remote
def find_links_task(start_url, base_url, depth=2):
    return find_links(start_url, base_url, depth)

base = "https://docs.ray.io/en/latest/"
docs = base + "index.html"

links = [find_links_task.remote(f"{base}{lib}/index.html",
                                base)
          for lib in ["", "rllib", "tune", "serve"]]

for res in ray.get(links):
    print(len(res))
```

[Speed up your web crawler by parallelizing it with Ray — Ray 2.46.0](#)

Using Spark on Ray (RayDP)

- RayDP combines the Spark and Ray clusters
 - Do large scale data processing using PySpark API
 - Then use to train the models with PyTorch

```
import ray
import raydp

ray.init()
spark = raydp.init_spark(
    app_name = "example", num_executors = 10,
    executor_cores = 64, executor_memory = "256GB"
)
from pyspark.sql.functions import col
df = spark.range(1, 1000)
# calculate z = x + 2y + 1000
df = df.withColumn("x", col("id")*2)\
        .withColumn("y", col("id") + 200)\
        .withColumn("z", col("x") + 2*col("y") + 1000)

from raydp.utils import random_split
train_df, test_df = random_split(df, [0.7, 0.3])
```

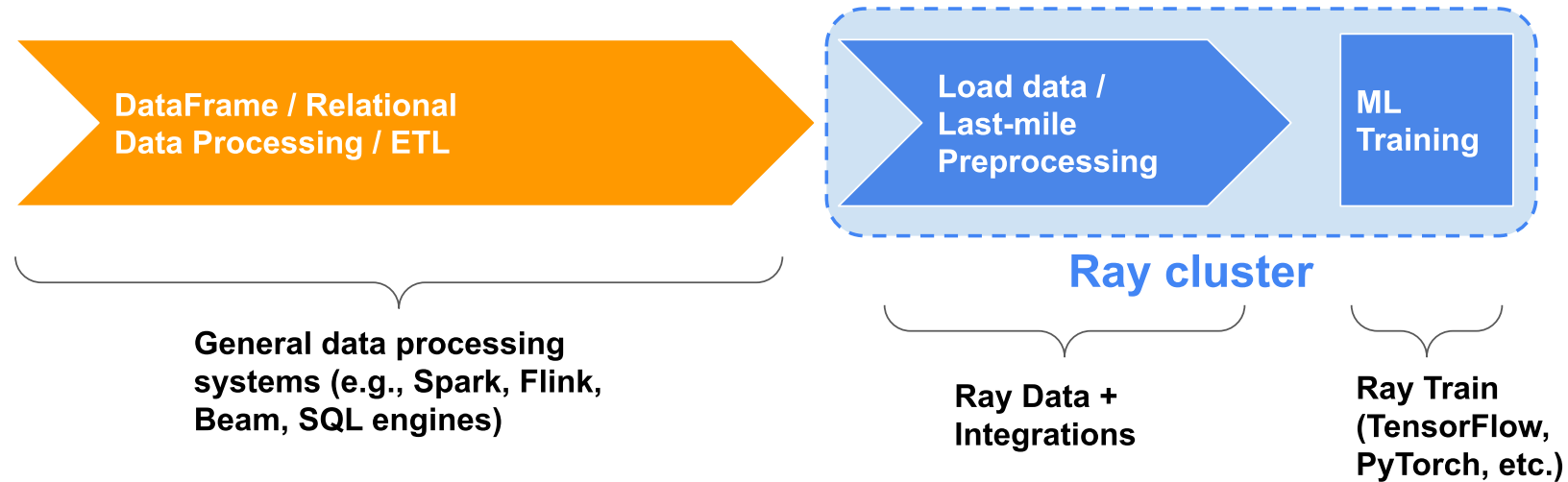
```
# PyTorch Code
import torch
model = LinearModel()
optimizer = torch.optim.Adam(model.parameters())
loss_fn = torch.nn.MSELoss()

def lr_scheduler_creator(optimizer, config):
    return torch.optim.lr_scheduler.MultiStepLR(
        optimizer, milestones=[150, 250, 350], gamma=0.1)

from raydp.torch import TorchEstimator
estimator = TorchEstimator(
    num_workers = 2,
    model = model,
    optimizer = optimizer,
    loss = loss_fn,
    lr_scheduler_creator=lr_scheduler_creator,
    feature_columns = ["x", "y"],
    label_column = ["z"],
    batch_size = 1000,
    num_epochs = 2
)
estimator.fit_on_spark(train_df, test_df)
pytorch_model = estimator.get_model()
```

[Using Spark on Ray \(RayDP\) — Ray 2.46.0](#)

- A scalable data processing library for ML workloads
- Use **streaming** execution to efficiently process large datasets
- Offline batch inference
 - A process for generating model predictions on **a fixed set** of input data
- Data preprocessing and ingest for ML training



Case Studies for Ray Data batch inference

Overview	
Getting Started	
Installation	
Use Cases	▼
Example Gallery	
Ecosystem	
Ray Core	▼
Ray Data	▼
Ray Data Quickstart	
Key Concepts	
User Guides	▼
Examples	
Ray Data API	▼
Comparing Ray Data to other systems	
Ray Data Internals	
Ray Train	▼
Ray Tune	▼
Ray Serve	▼

🏠 > Ray Data:...

Ray Data: Scalable Datasets for ML

Ray Data is a scalable data processing library for ML and AI workloads built on Ray. F and ingest for ML training. Unlike other distributed data systems, Ray Data features a workloads.

Why choose Ray Data?

Modern AI workloads revolve around the usage of deep learning models, which are c memory, have different semantics for scheduling, and are much more expensive to r

Ray Data supports AI workloads as a first-class citizen and offers several key advant:

- **Faster and cheaper for deep learning:** Ray Data streams data between CPU pre
- **Framework friendly:** Ray Data provides performant, first-class integration with c
- **Support for multi-modal data:** Ray Data leverages Apache Arrow and Pandas an and more.
- **Scalable by default:** Built on Ray for automatic scaling across heterogeneous clu hundreds of TB of data.

[Ray Data: Scalable Datasets for ML — Ray 2.46.0](#)

- Loading Data
 - Create datasets from on-disk files, Python objects, and cloud storage services like S3
- Transforming data
 - Apply user-defined functions (UDFs) to transform datasets
- Consuming data
 - Pass datasets to Ray Tasks or Actors, and access records with methods like `take_batch()` and `iter_batches()`.
- Saving data
 - Call methods like `write_parquet()` to save dataset contents to local or remote filesystems

High performance distributed IO

Read from storage

```
ds = ray.data.read_parquet("s3://some/bucket")  
ds = ray.data.read_csv("/tmp/some_file.csv")
```

Convert in memory

```
ds = ray.data.from_pandas(df)  
df = ds.to_pandas()  
ds = ray.data.from_dask(dask_df)  
dask_df = ds.to_dask()
```

Saving to storage

```
ds.write_parquet("s3://some/bucket")  
ds.write_csv("/tmp/some_file.csv")
```

- Compose transformations to express a chain of computations to process and modify your dataset

```
def parse_filename(row: Dict[str, Any]) ->
Dict[str, Any]:
    row["filename"] =
os.path.basename(row["path"])
    return row

ds = (ray.data.read_images("s3://anonymous@ray-
example-data/image-datasets/simple",
include_paths=True)
    .map(parse_filename))
```

```
class TorchPredictor:
    def __init__(self):
        self.model = torch.nn.Identity().cuda()
        self.model.eval()

    def __call__(self, batch: Dict[str, np.ndarray]) ->
Dict[str, np.ndarray]:
        inputs = torch.as_tensor(batch["data"],
dtype=torch.float32).cuda()
        with torch.inference_mode():
            batch["output"] =
self.model(inputs).detach().cpu().numpy()
        return batch

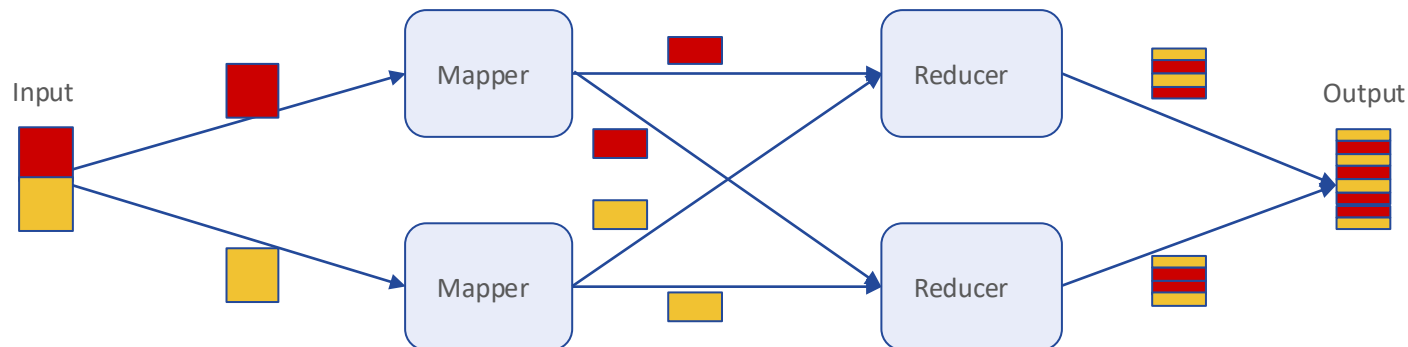
ds = (ray.data.from_numpy(np.ones((32, 100)))
    .map_batches(
        TorchPredictor,
        # Two workers with one GPU each
        concurrency=2,
        # Batch size is required if you're using GPUs.
        batch_size=4,
        num_gpus=1))
```

Last-Mile Preprocessing

- Basic transformations and aggregations
 - Map, batch map, filter
 - Stats aggregations
- Global shuffle operations
 - Sort
 - Random shuffle
 - Groupby

```
ray.data.read_parquet("foo.parquet") \
    .filter(lambda x: x < 0) \
    .map(lambda x: x**2) \
    .random_shuffle() \
    .write_parquet("bar.parquet")
```

```
ray.data.read_parquet("foo.parquet") \
    .map_batches(
        process_batch, batch_size=512) \
    .repartition(16) \
    .groupby("col_1").mean("col_2")
```



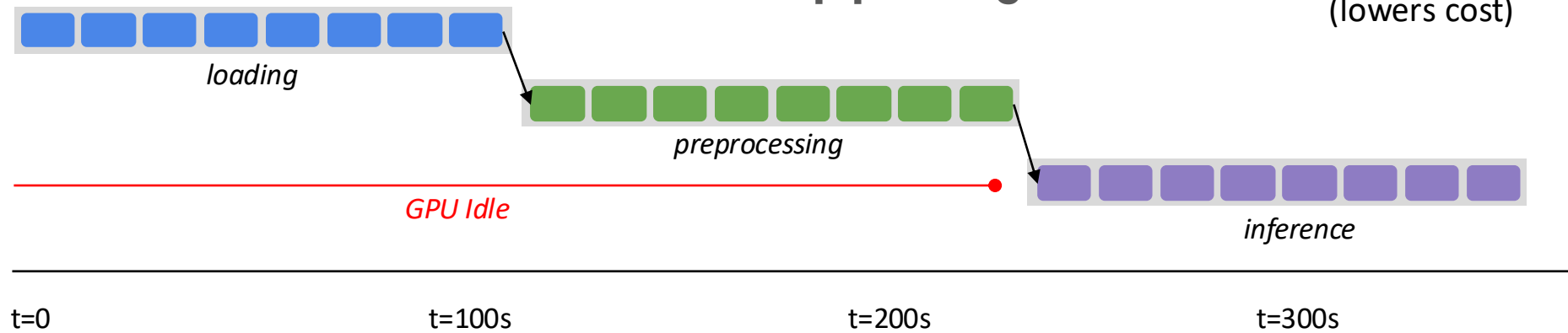
Pipelining CPU/GPU Compute



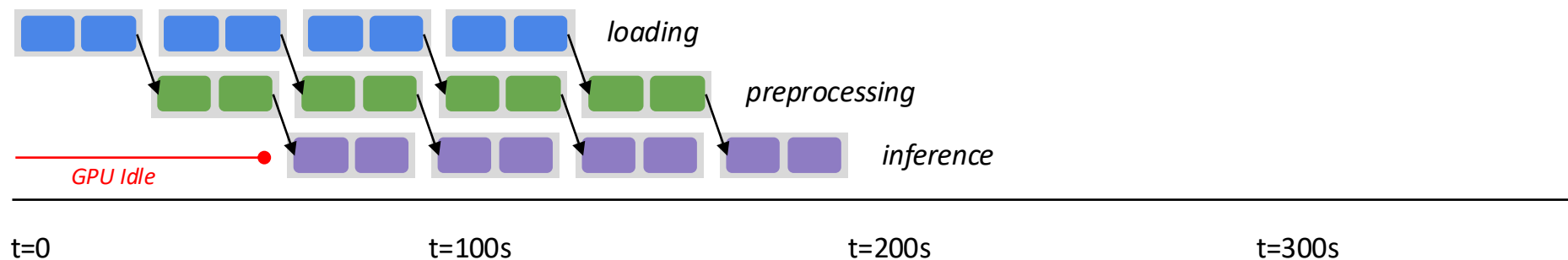
- Pipeline the execution of stages in your workflow

- Parallelizes stage execution (lower latency)
- Reduces idle time of stage-specific resources (lowers cost)

Without pipelining:



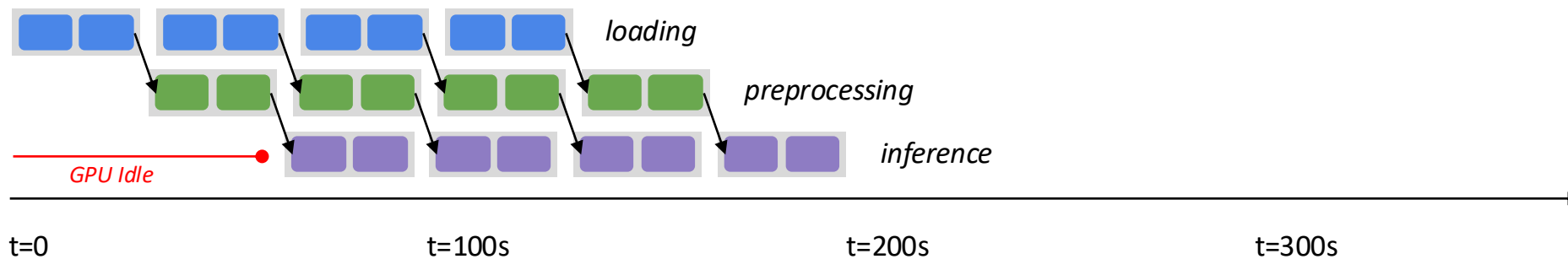
With pipelining:



Datasets solution - pipelining

- Pipelining of data loading, preprocessing, and inference stages

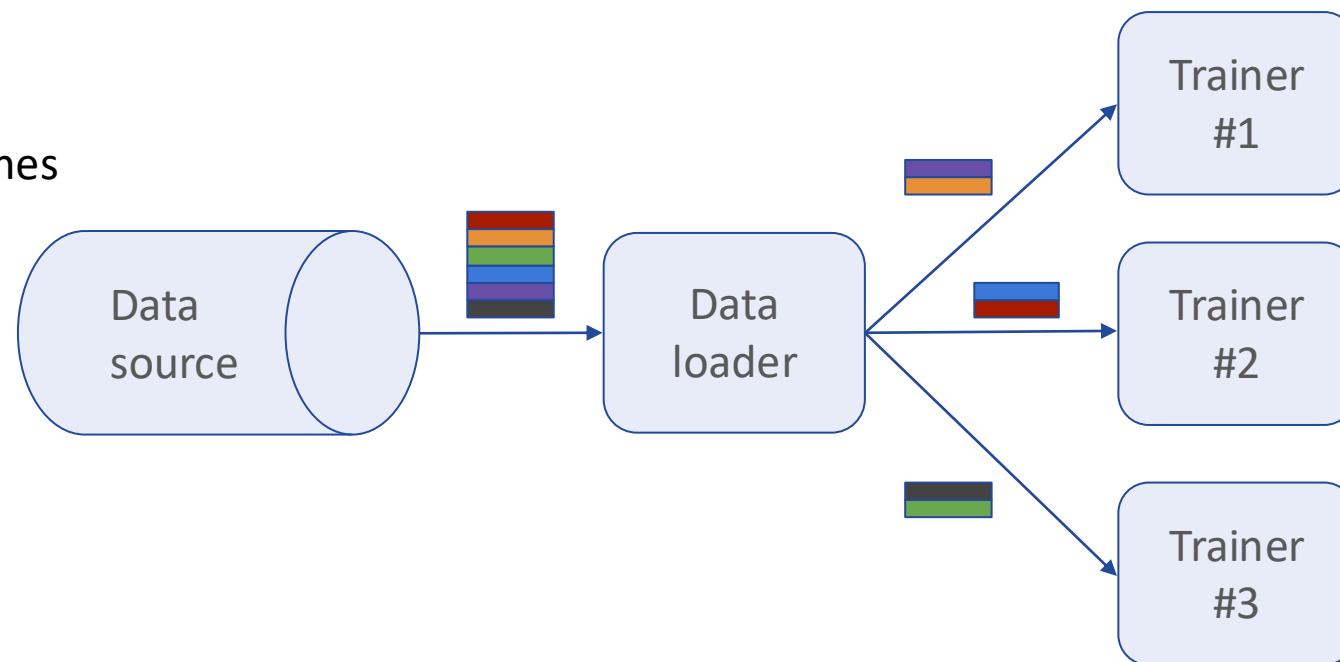
```
pipe: DatasetPipeline = ray.data \
    .read_binary_files("s3://bucket/image-dir") \
    .window(blocks_per_window=2) \
    .map(preprocess) \
    .map_batches(BatchInferModel, batch_size=256, num_gpus=1) \
    .write_json("/tmp/results")
```



Machine Learning Ingest



- Loading data into one or more model trainers
- Primary functions
 - Loading and Partitioning the data
 - Shuffling and batching the data into GPU batches
 - Local shuffling vs. global shuffling

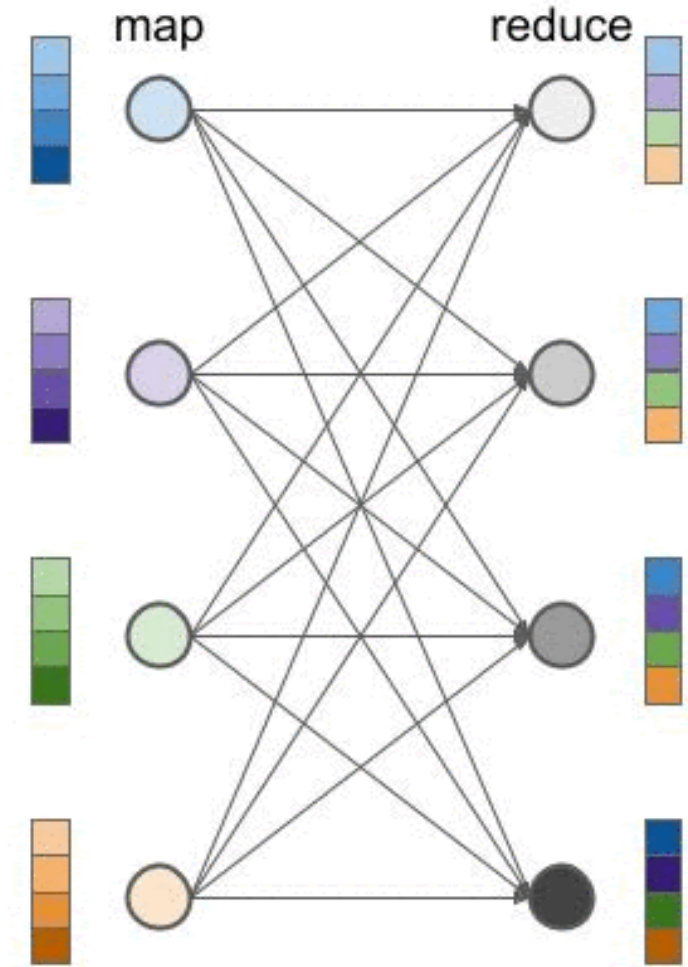


Why Shuffle Before Each Epoch?

- For certain models, per-epoch shuffling gives you much better model accuracy after a set number of epochs
- Shuffling decorrelates samples within and across batches
- Per-epoch shuffling reduces variance, improves generalization, and prevents overfitting

Why is Scalable Shuffled ML Ingest Hard?

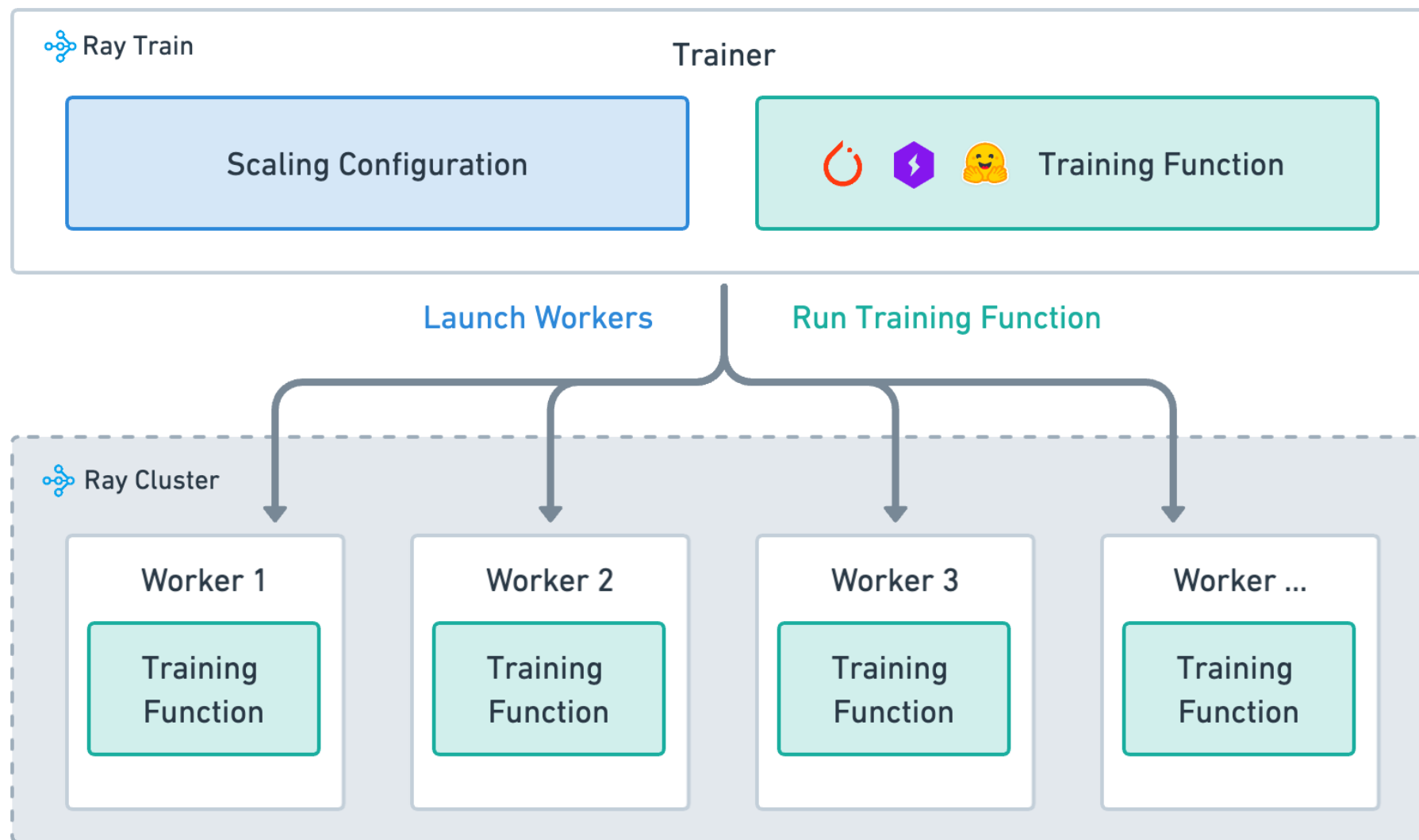
- The full pipeline may not fit in memory
- Local shuffles hurt model accuracy
- Global per-epoch shuffling difficult to do efficiently in a distributed setting



- A scalable machine learning library for distributed training and fine-tuning
- The training function is a user-defined Python function that contains the end-to-end model training loop logic.
- PyTorch Ecosystem
 - Pytorch, Pytorch lightning
 - Hugging Face Transformers/Accelerate
 - Microsoft Deepspeed
- Others: TensorFlow, Horovod, Keras

[Ray Train Overview — Ray 2.46.0](#)

Ray Train



- Training function
 - A user-defined Python function that contains the end-to-end model training loop logic
- ScalingConfig
 - Mechanism for defining the scale of the training job
- Trainer ties the previous three concepts together to launch distributed training jobs.

```
def train_func():  
    """User-defined training function that runs on each distributed worker process.  
  
    This function typically contains logic for loading the model,  
    loading the dataset, training the model, saving checkpoints,  
    and logging metrics.  
    """  
    ...
```

```
from ray.train import ScalingConfig  
  
# Single worker with a CPU  
scaling_config = ScalingConfig(num_workers=1, use_gpu=False)  
  
# Single worker with a GPU  
scaling_config = ScalingConfig(num_workers=1, use_gpu=True)  
  
# Multiple workers, each with a GPU  
scaling_config = ScalingConfig(num_workers=4, use_gpu=True)
```

- Example

```
from ray.train.torch import TorchTrainer
from ray.train import ScalingConfig

def train_func():
    # Your Transformers training code here.

scaling_config = ScalingConfig(num_workers=2, use_gpu=True)
trainer = TorchTrainer(train_func, scaling_config=scaling_config)
result = trainer.fit()
```

[Get Started with Distributed Training using Hugging Face Transformers — Ray 2.46.0](#)

Ray Environment Dependencies

- A runtime environment describes the dependencies your Ray application needs to run, including files, packages, environment variables, and more

```
import ray
import requests

# This example runs on a local machine, but you can also do
# ray.init(address=..., runtime_env=...) to connect to a cluster.
ray.init(runtime_env={"pip": ["requests"]})

@ray.remote
def reqs():
    return requests.get("https://www.ray.io/").status_code

print(ray.get(reqs.remote()))
```

Further Reading

- [Ray v2 Architecture - Google Docs](#)
- [Ray Educational Materials \(github\)](#)
- [text-generation-webui \(github.com\)](#)
- [vllm \(github.com\)](#)
- [lmdeploy \(github.com\)](#)
- [RISELab 分布式应用框架Ray项目介绍-云社区-华为云](#)



Thank you